UNIT-III

Basic Statistics (for Machine Learning)

1. Introduction

Basic Statistics forms the foundation of machine learning.
It helps in:

- Understanding data
- Summarizing information
- Making predictions
- Building probabilistic models

☞ Many ML algorithms (like Gaussian Mixture Models, PCA, etc.) rely on statistical concepts

2. Types of Data

- **Discrete Data** → Countable (e.g., number of students)
- **Continuous Data** → Real-valued (e.g., height, weight)

3. Measures of Central Tendency

These describe the **center of data**.

3.1 Mean (Average)

$$
\mu = \frac{1}{N} \sum_{i=1}^{N} x_i
$$

☞ Most commonly used measure

3.2 Median

- Middle value after sorting
- Less affected by outliers

3.3 Mode

- Most frequent value

4. Measures of Spread (Variability)

These describe how data is **spread out**.

4.1 Variance

$$
\sigma^2 = \frac{1}{N} \sum (x_i - \mu)^2
$$

☞ Measures dispersion from mean

4.2 Standard Deviation

$$
\sigma = \sqrt{\sigma^2}
$$

☞ Indicates spread in same units as data

Diagram

Low Variance: ••••• (points close)

High Variance: • • • •

5. Covariance

Measures how two variables change together:

$$
Cov(X, Y) = \frac{1}{N} \sum (x_i - \mu_x)(y_i - \mu_y)
$$

Interpretation

- Positive → variables increase together
- Negative → one increases, other decreases

- Zero → no relation

6. Correlation

Normalized version of covariance:

$$
\rho = \frac{Cov(X, Y)}{\sigma_x \sigma_y}
$$

Range:

$$
-1 \leq \rho \leq 1
$$

Diagram

Positive Correlation:

Negative Correlation:

No Correlation: random points

7. Probability Basics

Probability measures likelihood of an event:

$$
0 \leq P(A) \leq 1
$$

Conditional Probability

$$
P(A|B) = \frac{P(A \cap B)}{P(B)}
$$

8. Gaussian (Normal) Distribution ⭑

One of the most important distributions in ML.

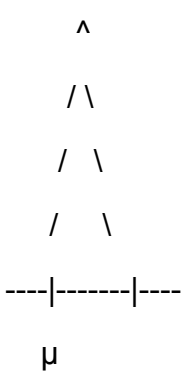
Formula

$$
P(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}
$$

Properties

- Bell-shaped curve
- Symmetric around mean
- Defined by:
    - Mean (μ)
    - Variance (σ²)

Diagram



9. Importance in Machine Learning

Basic statistics is used in:

- Data preprocessing
- Feature scaling
- Probability models
- Dimensionality reduction
- Clustering algorithms

Gaussian Mixture Models (GMM)

1. Definition

A **Gaussian Mixture Model (GMM)** is a probabilistic model that represents data as a **mixture of multiple Gaussian (normal) distributions**.

☞ Each Gaussian represents a **cluster** in the data

2. Key Idea

Instead of assuming data comes from a single distribution:

☞ Assume data is generated from **K different Gaussian distributions**

[
P(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x | \mu_k, \Sigma_k)
]

Where:

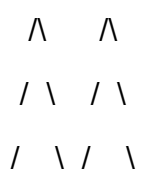
- (\pi_k) → mixing coefficient
- (\mu_k) → mean
- (\Sigma_k) → covariance

3. Intuition

- Data points belong to clusters
- Each cluster has its own Gaussian distribution
- A point has **probability of belonging to each cluster**

Diagram

Data Distribution:



Cluster1 Cluster2

(Gaussian curves)

4. Components of GMM

4.1 Mean (μ)

- Center of Gaussian

4.2 Covariance (Σ)

- Shape and spread

4.3 Mixing Coefficient (π)

[
\sum_{k=1}^{K} \pi_k = 1
]

- Weight of each Gaussian

5. Soft Clustering

Unlike K-means:

☞ GMM assigns **probabilities**, not fixed labels

Example:

- Point belongs:
    - 70% to Cluster 1
    - 30% to Cluster 2

6. Expectation-Maximization (EM) Algorithm ⋆

Used to estimate parameters of GMM.

Step 1: Initialize parameters

- Means, covariances, mixing coefficients

Step 2: Expectation (E-step)

Compute probability of each point belonging to each cluster:

[

\gamma_{ik} = P(z_k | x_i)
]

Step 3: Maximization (M-step)

Update parameters:

- Means
- Covariances
- Mixing coefficients

Diagram

Initialize Parameters

↓

E-step (Compute Probabilities)

↓

M-step (Update Parameters)

↓

Repeat until convergence

7. Difference from K-Means

GMM	**K-Means**
Soft clustering	Hard clustering
Uses probability	Uses distance
Elliptical clusters	Spherical clusters
More flexible	Simpler

8. Applications

- Clustering
- Image segmentation

- Speech recognition
- Anomaly detection

9. Advantages

- Flexible cluster shapes
- Probabilistic interpretation
- Handles overlapping clusters

10. Limitations

- Computationally expensive
- Sensitive to initialization
- May converge to local optima

Nearest Neighbor Methods

1. Definition

Nearest Neighbor Methods are simple, instance-based learning techniques that classify or predict the value of a data point based on the **closest data points in the training set**.

☞ Idea:

- Similar inputs → similar outputs

2. Key Concept

Given a new data point:

☞ Find the **nearest (closest) training examples**
☞ Use them to make prediction

3. Types of Nearest Neighbor Methods

3.1 K-Nearest Neighbors (KNN)

- Uses **K closest points**
- Majority voting (classification)
- Average (regression)

3.2 1-Nearest Neighbor

- Uses only the closest point

4. Distance Measures ⭐

To find nearest neighbors, we need a **distance metric**:

4.1 Euclidean Distance

[
d(x, y) = \sqrt{\sum (x_i - y_i)^2}
]

4.2 Manhattan Distance

[
d(x, y) = \sum |x_i - y_i|
]

4.3 Other Distances

- Minkowski distance
- Cosine similarity

5. Working of KNN

Steps:

1. Choose value of K

2. Compute distance from test point to all training points
3. Select K nearest neighbors
4. Predict:
    - Classification → majority vote
    - Regression → average

Diagram

▲

● ● ● ▲ ← New Point

Neighbors:

● ● ● (majority class)

6. Decision Boundary

- KNN creates **non-linear decision boundaries**
- Depends on data distribution

Diagram

Class ● and ▲ mixed

Decision boundary becomes irregular

7. Nearest Neighbor Smoothing

Used in regression:

$$
\hat{y} = \frac{1}{K} \sum y_i
$$

☞ Average of neighbors

8. Efficient Computation (KD-Tree)

Problem:

- Distance computation is expensive

Solution:

- Use **KD-Tree** for faster search

Diagram

Space divided into regions

(Search becomes faster)

9. Advantages

- Simple and easy to implement
- No training phase (lazy learning)
- Works well for small datasets

10. Limitations

- Slow for large datasets
- Sensitive to noise
- Choice of K is critical
- Affected by irrelevant features

11. Applications

- Classification problems
- Recommendation systems
- Pattern recognition
- Image classification

Unsupervised Learning

1. Definition

Unsupervised Learning is a type of machine learning where:

☞ The model is trained **without labeled data**
☞ It tries to **discover patterns, structures, or relationships** in the data

2. Key Idea

- No target/output labels are given
- The algorithm groups or organizes data based on similarity

3. Types of Unsupervised Learning

- **Clustering** → Group similar data points
- **Dimensionality Reduction** → Reduce features
- **Density Estimation** → Model data distribution

4. Examples

- Customer segmentation
- Image grouping
- Anomaly detection

5. Characteristics

- No supervision (no correct answers)
- Exploratory in nature
- Finds hidden patterns

6. Challenges

- Hard to evaluate results
- Sensitive to data quality
- Interpretation can be difficult

7. Summary

- Unsupervised learning finds structure in data
- Does not require labeled outputs
- Important for clustering and data analysis

K-Means Algorithm

1. Definition

K-Means is a popular unsupervised learning algorithm used for **clustering** data into K groups.

☞ Each cluster is represented by its **centroid (mean)**

2. Key Idea

- Divide data into K clusters
- Minimize distance between data points and their cluster center

3. Objective Function

[
J = \sum_{k=1}^{K} \sum_{x_i \in C_k} ||x_i - \mu_k||^2
]

Where:

- (\mu_k) → centroid of cluster k

4. Algorithm Steps

1. Choose number of clusters K
2. Initialize K centroids randomly

3. Assign each point to nearest centroid
4. Update centroids (mean of cluster points)
5. Repeat steps 3–4 until convergence

5. Working Diagram

Initial Points:

● ● ● ▲ ▲ ▲

Step 1: Assign clusters

Step 2: Compute centroids

Step 3: Reassign points

Repeat until stable clusters

6. Iterative Process

Initialize Centroids

↓

Assign Points to Clusters

↓

Update Centroids

↓

Repeat until no change

7. Example

- K = 2
- Data split into 2 groups
- Each group forms a cluster

8. Properties

- Hard clustering (each point belongs to one cluster)
- Uses Euclidean distance
- Fast and efficient

9. Choosing K

Methods:

- Elbow method
- Silhouette score

10. Advantages

- Simple and easy to implement
- Fast for large datasets
- Works well for spherical clusters

11. Limitations

- Requires predefined K
- Sensitive to initial centroids
- Not suitable for non-spherical clusters
- Affected by outliers

12. Applications

- Customer segmentation
- Image compression
- Pattern recognition
- Market analysis

UNIT-IV

Dimensionality Reduction

1. Definition

Dimensionality Reduction is the process of reducing the number of input features (dimensions) in a dataset while preserving the most important information.

2. Motivation

Real-world datasets often have **high dimensions**, which leads to several problems:

- Increased computational cost
- Need for large training data
- Difficulty in visualization
- Overfitting in models

This phenomenon is known as the **curse of dimensionality**

3. Why Dimensionality Reduction?

Dimensionality reduction is useful because it:

- Reduces noise in data
- Improves model performance
- Simplifies data representation
- Enables visualization in 2D or 3D
- Reduces storage and computation

4. Types of Dimensionality Reduction

4.1 Feature Selection

- Select a subset of original features
- Remove irrelevant or redundant features

4.2 Feature Extraction (Transformation)

- Transform data into a lower-dimensional space
- Combine features to form new ones

Examples:

- PCA
- LDA
- ICA

4.3 Clustering-Based Methods

- Group similar data points
- Represent clusters instead of individual data

5. Basic Concept

We transform data from high dimension to low dimension:

$$
X \in \mathbb{R}^M \rightarrow Y \in \mathbb{R}^L \quad (L < M)
$$

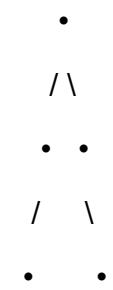
6. Geometric Interpretation

Dimensionality reduction can be seen as:

- Rotating the coordinate system
- Projecting data onto a lower-dimensional space

Diagram

Original Data (3D)



↓ Projection

Reduced Data (2D)

7. Example Insight

Sometimes data looks complex in original space but simple in transformed space.

Example:

- Points in (x, y) appear scattered
- When transformed → lie on a circle (single variable: angle)

8. Methods Classification

Type	Methods
Supervised	LDA
Unsupervised	PCA, ICA, LLE, Isomap
Statistical	Factor Analysis

9. Key Goals

- Preserve important structure
- Retain maximum variance or information
- Reduce redundancy
- Improve learning efficiency

10. Challenges

- Loss of information
- Choosing optimal dimensions
- Maintaining interpretability

Linear Discriminant Analysis (LDA)

1. Definition

Linear Discriminant Analysis (LDA) is a **supervised dimensionality reduction technique** used to project data onto a lower-dimensional space such that **different classes are well separated**.

2. Key Idea

LDA aims to:

☞ **Maximize distance between classes**
☞ **Minimize spread within each class**

In simple terms:

- Make classes far apart
- Make points within same class close together

3. Intuition

Consider two classes of data points:

- If projected poorly → classes overlap
- If projected correctly → classes are clearly separable

Diagram (Projection Concept)

Original Data (2D)

Class 1: ● ● ●

Class 2: ▲ ▲ ▲

Bad Projection → Overlap

Good Projection → Separation

4. Scatter Matrices (Core Concept ⭐)

LDA uses two types of scatter:

4.1 Within-Class Scatter (S_x)

Measures how spread out points are within each class:

$$
S_W = \sum_{c} \sum_{x \in c} (x - \mu_c)(x - \mu_c)^T
$$

✓ Should be **small**

4.2 Between-Class Scatter (S_b)

Measures distance between class means:

$$
S_B = \sum_{c} (\mu_c - \mu)(\mu_c - \mu)^T
$$

✓ Should be **large**

Diagram

Class 1 → tight cluster

Class 2 → tight cluster

Distance between means → large (good separation)

5. Objective Function

LDA finds projection vector **w** that maximizes:

$$
J(w) = \frac{w^T S_B w}{w^T S_W w}
$$

☞ Maximize between-class variance
☞ Minimize within-class variance

6. Projection

Data is projected as:

$$
z = w^T x
$$

Diagram

Original Data (2D)



↓ Projection onto w

1D Axis:



7. Solution

For two-class problem:

$$
w = S_W^{-1} (\mu_1 - \mu_2)
$$

For multi-class:

- Use eigenvectors of (S_W^{-1} S_B)

8. Algorithm Steps

1. Compute class means (\mu_c)
2. Compute within-class scatter (S_W)
3. Compute between-class scatter (S_B)
4. Solve eigenvalue problem
5. Select top eigenvectors
6. Project data

9. Properties

- Supervised method
- Uses class labels
- Maximizes class separability
- Output dimensions ≤ (number of classes − 1)

10. Applications

- Face recognition
- Pattern classification
- Medical diagnosis
- Image recognition

11. Advantages

- Improves class separability
- Reduces dimensionality
- Works well for classification tasks

12. Limitations

- Assumes linear separability
- Sensitive to outliers
- Requires labeled data

Principal Component Analysis (PCA)

1. Definition

Principal Component Analysis (PCA) is an **unsupervised dimensionality reduction technique** that transforms data into a new coordinate system such that:

☞ The **maximum variance** lies along the first axis (principal component)
☞ Remaining variance lies along subsequent orthogonal axes

2. Key Idea

PCA finds new axes (directions) such that:

- First axis → captures maximum variance
- Second axis → captures next maximum variance
- And so on…

These axes are called **principal components**

3. Intuition

Instead of using original axes:

☞ Rotate the coordinate system
☞ Align it with direction of maximum spread

Diagram (Concept)

Original Data (Tilted Ellipse)

-



Axes not aligned with data

↓

After PCA (Rotated Axes)

• • • • • → PC1 (max variance)

|

| PC2 (less variance)

4. Steps in PCA

1. **Standardize data** (mean = 0)
2. Compute covariance matrix
3. Compute eigenvalues & eigenvectors
4. Sort eigenvectors (by eigenvalues)
5. Select top components
6. Project data

5. Mathematical Formulation

Let data matrix be (X)

Covariance Matrix:

[
C = \frac{1}{N} X^T X
]

Eigenvalue Equation:

[
C p_i = \lambda_i p_i
]

Where:

- (p_i) = eigenvector (principal component)
- (\lambda_i) = eigenvalue (variance along that direction)

6. Projection

Data is transformed as:

[
Y = P^T X
]

Where:

- (P) = matrix of eigenvectors

7. Interpretation

- Large eigenvalue → important direction
- Small eigenvalue → less important → can be removed

8. Dimensionality Reduction

Select top **k eigenvectors**:

High-dimensional data → PCA → Reduced dimensions

(x1, x2, x3, x4) → (PC1, PC2)

9. Geometric View

PCA:

- Rotates data
- Aligns axes with data spread
- Removes low-variance directions

10. Properties

- Unsupervised method
- Components are orthogonal
- Removes correlation between features
- Maximizes variance

11. Applications

- Data compression
- Noise reduction
- Image processing
- Visualization
- Preprocessing for ML models

12. Advantages

- Simple and efficient
- Reduces redundancy
- Improves performance
- Easy to implement

13. Limitations

- Assumes linear relationships
- Sensitive to scaling
- Does not use class labels
- May lose interpretability

Factor Analysis

1. Definition

Factor Analysis (FA) is a statistical dimensionality reduction technique used to explain observed variables in terms of a **smaller number of unobserved variables called factors (latent variables)**.

☞ Goal:

- Reduce dimensions
- Identify hidden structure in data

2. Key Idea

Observed data is assumed to be generated from:

☞ A few **latent factors**
☞ Plus some **noise**

[
X = W Y + E
]

Where:

- (X) → observed data
- (Y) → latent factors
- (W) → factor loadings
- (E) → noise

3. Intuition

Instead of treating all features as independent:

☞ Assume they are influenced by hidden causes

Example

- Exam scores in subjects:
    - Math, Physics, Chemistry

☞ Influenced by:

- Intelligence (latent factor)
- Study habits

4. Model Representation

Diagram

Latent Factors (Y)

↓

[W matrix]

↓

Observed Variables (X)

↓

Noise (E)

5. Assumptions

- Latent factors are **independent**
- Noise is:
    - Gaussian
    - Mean = 0
- Observed variables are linear combinations of factors

6. Covariance Structure

Covariance of data is:

[
\Sigma = W W^T + \Psi
]

Where:

- (W W^T) → explained variance
- (\Psi) → noise variance

7. Goal of Factor Analysis

Find:

- Factor loadings (W)

- Noise variance (\Psi)

Such that data can be reconstructed

8. Difference from PCA

Factor Analysis	PCA
Models noise explicitly	No noise model
Based on probabilistic model	Based on variance
Latent variables meaningful	Components may not be interpretable

9. Algorithm (EM Approach)

Factor Analysis is typically solved using **Expectation-Maximization (EM)**:

Steps:

1. Initialize parameters (W, \Psi)
2. **E-step**: Estimate latent variables
3. **M-step**: Update parameters
4. Repeat until convergence

10. Applications

- Psychology (IQ analysis)
- Social sciences
- Finance (risk factors)
- Bioinformatics

11. Advantages

- Identifies hidden structure
- Reduces dimensionality
- Handles noise explicitly

12. Limitations

- Assumes linear relationships
- Requires estimation of parameters
- Sensitive to model assumptions

Independent Component Analysis (ICA)

1. Definition

Independent Component Analysis (ICA) is a dimensionality reduction technique used to **separate a multivariate signal into independent non-Gaussian components**.

☞ Goal:

- Find underlying **independent sources** from observed data

2. Key Idea

Observed data is assumed to be a **mixture of independent signals**:

[
X = A S
]

Where:

- (X) → observed signals
- (S) → independent source signals (unknown)
- (A) → mixing matrix

☞ ICA tries to recover:
[
S = W X
]

Where (W = A^{-1})

3. Intuition (Blind Source Separation)

Classic example:

☞ **Cocktail Party Problem**

- Multiple people speaking
- Multiple microphones record mixed signals
- ICA separates individual voices

Diagram

Source Signals (S1, S2)

↓

Mixing Process (A)

↓

Observed Signals (X1, X2)

↓

ICA Algorithm

↓

Recovered Sources (Ŝ1, Ŝ2)

4. Assumptions

- Source signals are **statistically independent**
- Signals are **non-Gaussian**
- Mixing is **linear**

5. Key Principle

ICA maximizes **statistical independence** between components.

☞ Uses:

- Non-Gaussianity
- Higher-order statistics

6. Steps in ICA

1. Center the data (mean = 0)
2. Whiten the data (remove correlations)
3. Find independent components by maximizing independence
4. Compute unmixing matrix (W)
5. Recover sources

7. Difference from PCA and FA

Method	Goal
PCA	Maximize variance
Factor Analysis	Model latent variables + noise
ICA	Maximize independence

8. Geometric Interpretation

- PCA → finds orthogonal directions
- ICA → finds directions that make data statistically independent

9. Applications

- Signal processing (audio separation)
- EEG/brain signal analysis
- Image processing
- Financial data analysis

10. Advantages

- Can recover hidden independent signals
- Useful for blind source separation
- Works beyond second-order statistics

11. Limitations

- Requires non-Gaussian data
- Cannot determine order of components
- Sensitive to noise

Locally Linear Embedding (LLE)

1. Definition

Locally Linear Embedding (LLE) is a **non-linear dimensionality reduction technique** that preserves the **local structure of data**.

☞ It assumes:

- Data lies on a **low-dimensional manifold**
- Each point can be reconstructed from its **nearest neighbors**

2. Key Idea

Instead of preserving global distances:

☞ LLE preserves **local relationships between points**

Steps:

1. Find nearest neighbors
2. Express each point as a linear combination of neighbors
3. Map data to lower dimension while preserving these relationships

3. Intuition

- High-dimensional data may lie on a curved surface (manifold)
- LLE “unwraps” this surface into lower dimensions

Diagram (Manifold to Low Dimension)

High-Dimensional Manifold (Curved Surface)



↓

Flattened Representation (Low Dimension)

4. Algorithm Steps

Step 1: Find Nearest Neighbors

For each data point:

- Identify k nearest neighbors

Step 2: Compute Reconstruction Weights

Each point is reconstructed from neighbors:

$$
x_i = \sum_j w_{ij} x_j
$$

Where:

- w_{ij} = weight of neighbor j

Constraint:

$$
\sum_j w_{ij} = 1
$$

Step 3: Compute Low-Dimensional Embedding

Find new points y_i such that:

$$
y_i = \sum_j w_{ij} y_j
$$

☞ Preserve same weights in low dimension

5. Objective Function

Minimize reconstruction error:

$$
\sum_i \left| x_i - \sum_j w_{ij} x_j \right|^2
$$

6. Key Properties

- Preserves **local geometry**
- Non-linear technique
- Does not preserve global distances

7. Advantages

- Captures non-linear structure
- Works well for manifold learning
- No need for distance preservation globally

8. Limitations

- Sensitive to noise
- Choice of neighbors (k) is critical
- Computationally expensive
- Not good for sparse data

9. Comparison with PCA

LLE	PCA
Non-linear	Linear

Preserves local structure	Preserves global variance
Works on manifolds	Works on flat data

10. Applications

- Image processing
- Face recognition
- Data visualization
- Manifold learning

Isomap (Isometric Mapping)

1. Definition

Isomap (Isometric Mapping) is a **non-linear dimensionality reduction technique** that preserves the **global geometric structure** of data.

☞ It extends **Multidimensional Scaling (MDS)** by using **geodesic distances** instead of Euclidean distances.

2. Key Idea

- Data lies on a **curved manifold**
- Straight-line (Euclidean) distance is misleading
- Use **shortest path distance along the manifold (geodesic distance)**

3. Intuition

Imagine points lying on a curved surface:

- Euclidean distance → cuts through the surface (wrong)
- Geodesic distance → follows the surface (correct)

Diagram (Concept)

Curved Surface:

A • -------- • B (Euclidean: straight line ✗)

A • ~~~~~~~~ • B (Geodesic: along surface ✓)

4. Algorithm Steps

Step 1: Construct Neighborhood Graph

- Connect each point to its k nearest neighbors

Step 2: Compute Geodesic Distances

- Use shortest path algorithm (e.g., Dijkstra)
- Compute distances between all pairs

Step 3: Apply MDS (Multidimensional Scaling)

- Use distance matrix
- Project data into lower dimensions

5. Process Diagram

High-Dimensional Data

↓

Construct Neighbor Graph

↓

Compute Shortest Paths (Geodesic Distances)

↓

Apply MDS

↓

Low-Dimensional Embedding

6. Key Properties

- Preserves **global structure**
- Works on **non-linear manifolds**
- Uses graph-based distances

7. Difference from LLE

Isomap	LLE
Preserves global geometry	Preserves local structure
Uses geodesic distance	Uses local linear weights
Graph + shortest path	Neighbor reconstruction

8. Advantages

- Captures global structure of data
- Effective for curved manifolds
- Provides meaningful low-dimensional representation

9. Limitations

- Sensitive to noise
- Computationally expensive
- Choice of neighbors (k) is important
- May fail if graph is not well connected

10. Applications

- Data visualization
- Image processing
- Manifold learning
- Pattern recognition

Least Squares Optimization Evolutionary Learning

1. Definition

Least Squares Optimization is a mathematical method used to find the best-fitting solution by **minimizing the sum of squared errors** between observed values and predicted values.

2. Key Idea

Given:

- Observed data (y)
- Predicted values (\hat{y})

Minimize:

[
\text{Error} = \sum (y_i - \hat{y}_i)^2
]

☞ Goal: Find parameters that minimize this error

3. Intuition

- Measure how far predictions are from actual values
- Square the errors to avoid cancellation
- Choose parameters that give **smallest total error**

4. Linear Model Representation

[
y = Xw + \epsilon
]

Where:

- (X) → input features
- (w) → parameters (weights)
- (\epsilon) → error

5. Objective Function

[
J(w) = ||y - Xw||^2
]

6. Solution

Optimal weights:

[
w = (X^T X)^{-1} X^T y
]

7. Diagram

Data Points:

-
-
-
-

Best Fit Line:

Goal: Minimize vertical distances (errors)

8. Applications

- Linear regression

- Curve fitting
- Signal processing
- Data modeling

9. Advantages

- Simple and efficient
- Closed-form solution available
- Widely used in ML

10. Limitations

- Sensitive to outliers
- Assumes linear relationship

Evolutionary Learning

1. Definition

Evolutionary Learning is a type of optimization inspired by **biological evolution**, where solutions evolve over time using mechanisms like:

- Selection
- Crossover
- Mutation

2. Key Idea

- Maintain a **population of solutions**
- Iteratively improve them
- Use **fitness function** to select best solutions

3. Basic Process

Initial Population

↓

Evaluate Fitness

↓

Selection

↓

Crossover + Mutation

↓

New Population

↓

Repeat

4. Genetic Algorithms (GA)

Definition

A **Genetic Algorithm** is an evolutionary technique that uses principles of genetics to solve optimization problems.

5. Components of Genetic Algorithm

5.1 Chromosome

- Representation of a solution

5.2 Population

- Set of possible solutions

5.3 Fitness Function

- Measures quality of solution

6. Genetic Offspring

New solutions are generated from parent solutions.

Diagram

Parent 1: 11001

Parent 2: 10110

↓ Crossover

Child 1: 11010

Child 2: 10101

7. Genetic Operators

7.1 Selection

- Choose best individuals based on fitness

7.2 Crossover

- Combine two parents to create offspring

7.3 Mutation

- Randomly change some bits

Diagram

Before Mutation: 11001

After Mutation: 11101

8. Using Genetic Algorithms

Steps

1. Initialize population
2. Evaluate fitness

3. Select parents
4. Apply crossover
5. Apply mutation
6. Form new generation
7. Repeat until convergence

9. Applications

- Optimization problems
- Feature selection
- Scheduling problems
- Neural network training

10. Advantages

- Works for complex problems
- Does not require gradient
- Global search capability

11. Limitations

- Computationally expensive
- May converge slowly
- Requires parameter tuning

12. Summary

- Least Squares → minimizes error in prediction
- Evolutionary Learning → optimization using natural selection
- Genetic Algorithms → key method in evolutionary learning

Genetic Algorithms (GA)

1. Definition

A **Genetic Algorithm (GA)** is a search and optimization technique inspired by the process of **natural selection and genetics**.

☞ It evolves a population of solutions over generations to find the **optimal or near-optimal solution**.

2. Key Idea

- Start with a set of possible solutions (**population**)
- Evaluate how good each solution is (**fitness**)
- Select the best ones
- Generate new solutions using genetic operations
- Repeat until optimal solution is found

3. Basic Terminology

Term	Meaning
Chromosome	A solution (encoded form)
Gene	Part of a chromosome
Population	Set of solutions
Fitness Function	Measures quality of solution
Generation	One iteration of evolution

4. Working Process

Diagram

Initial Population

↓

Evaluate Fitness

↓

Selection

↓

Crossover

↓

Mutation

↓

New Generation

↓

Repeat

5. Representation of Chromosome

Solutions are encoded as:

- Binary strings (most common)
- Real numbers
- Strings or arrays

Example

Chromosome: 101101

Gene: 1 0 1 1 0 1

6. Steps in Genetic Algorithm

1. Initialize population randomly
2. Evaluate fitness of each individual
3. Select parents based on fitness
4. Apply crossover to create offspring
5. Apply mutation to introduce variation
6. Replace old population
7. Repeat until stopping condition

7. Genetic Operators

7.1 Selection

- Chooses best individuals
- Methods:
    - Roulette Wheel
    - Tournament Selection

7.2 Crossover (Recombination)

- Combines two parents to produce offspring

Diagram

Parent 1: 11001

Parent 2: 10110

↓

Child 1: 11010

Child 2: 10101

7.3 Mutation

- Randomly alters genes to maintain diversity

Diagram

Before: 11001

After: 11101

8. Fitness Function

- Evaluates how good a solution is
- Guides the evolution process

Example:
[
f(x) = x^2
]

9. Advantages

- Works for complex and non-linear problems
- Does not require gradient information
- Can escape local optima
- Flexible and robust

10. Limitations

- Computationally expensive
- Slow convergence
- Requires parameter tuning (mutation rate, population size)

11. Applications

- Optimization problems
- Feature selection
- Scheduling
- Machine learning parameter tuning
- Robotics

Genetic Offspring – Genetic Operators – Using Genetic Algorithms

1. Genetic Offspring

1.1 Definition

Genetic offspring are new solutions generated from existing solutions (**parents**) during the evolution process in a Genetic Algorithm.

☞ Offspring inherit characteristics from parents and may also include variations.

1.2 Key Idea

- Combine good features of parents

- Introduce diversity
- Improve overall population quality

Diagram

Parent 1: 11001

Parent 2: 10110

↓

Offspring 1: 11010

Offspring 2: 10101

1.3 Importance

- Helps explore new solutions
- Avoids stagnation
- Drives evolution toward optimal solution

2. Genetic Operators

Genetic operators are mechanisms used to generate offspring and evolve the population.

2.1 Selection Operator

Definition

Selects the best individuals based on fitness.

Purpose

- Ensures better solutions are chosen for reproduction

Methods

- Roulette Wheel Selection
- Tournament Selection

2.2 Crossover Operator

Definition

Combines two parent chromosomes to produce offspring.

Diagram

Parent 1: 11001

Parent 2: 10110

Crossover Point ↓

Child 1: 11010

Child 2: 10101

Types of Crossover

- Single-point crossover
- Multi-point crossover
- Uniform crossover

2.3 Mutation Operator

Definition

Randomly alters one or more genes in a chromosome.

Diagram

Before Mutation: 11001

After Mutation: 11101

Purpose

- Introduces diversity
- Prevents premature convergence

3. Using Genetic Algorithms

3.1 General Procedure

1. Initialize population
2. Evaluate fitness
3. Select parents
4. Apply crossover
5. Apply mutation
6. Generate offspring
7. Replace population
8. Repeat until convergence

3.2 Flow Diagram

Start

↓

Initialize Population

↓

Evaluate Fitness

↓

Selection

↓

Crossover

↓

Mutation

↓

New Generation

↓

Stop? → No → Repeat

Yes → End

3.3 Key Considerations

- Population size
- Mutation rate
- Crossover probability
- Fitness function design

4. Advantages

- Can solve complex optimization problems
- Works without gradient information
- Flexible and adaptable

5. Limitations

- Computationally expensive
- May converge slowly
- Requires tuning of parameters

UNIT-V

Reinforcement Learning (RL) — Overview

1. Introduction

Reinforcement Learning (RL) is a type of machine learning where an agent learns how to behave in an environment by performing actions and receiving feedback in the form of rewards.

It lies between:

- Supervised Learning (has labeled data)
- Unsupervised Learning (no feedback)
- Reinforcement Learning (feedback but no correct answers)

The agent improves its performance through **trial-and-error learning**.

2. Key Concepts

2.1 Agent and Environment

- **Agent**: Learner or decision maker
- **Environment**: Everything the agent interacts with

2.2 State (s)

The current situation of the agent.

2.3 Action (a)

Possible moves the agent can take.

2.4 Reward (r)

Feedback signal received after taking an action.

2.5 Policy (π)

A strategy that defines what action to take in a given state:

$\pi(s) = a$

3. Reinforcement Learning Cycle

1. Agent observes current state (s_t)
2. Takes action (a_t)
3. Receives reward (r_{t+1})
4. Moves to next state (s_{t+1})

This process repeats continuously.

4. Objective of RL

The goal is to maximize cumulative reward over time:

$R_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + ...$

Where:

- γ (gamma) is the discount factor ($0 \leq \gamma \leq 1$)

5. Important Challenges

5.1 Delayed Reward

- Rewards may not be immediate
- Difficult to identify which action caused success

5.2 Exploration vs Exploitation

- **Exploration**: Trying new actions
- **Exploitation**: Using known best actions

A balance is required.

6. Reward Function

Defines the goal of the system:

$R(s, a) \rightarrow$ reward

Example:

- +100 → goal reached
- -1 → each step
- -100 → failure

A poorly designed reward function leads to incorrect behavior.

7. State Space and Action Space

State Space

All possible states the agent can be in.

Action Space

All possible actions the agent can take.

⚠ Large state space leads to **curse of dimensionality**.

8. Discount Factor (γ)

Used to reduce importance of future rewards:

- $\gamma \approx 0 \rightarrow$ focus on immediate reward
- $\gamma \approx 1 \rightarrow$ focus on long-term reward

9. Policy

A policy defines agent behavior:

- Deterministic policy: one action per state
- Stochastic policy: probability distribution over actions

Goal: find optimal policy π*

10. Value Functions

10.1 State Value Function

V(s): Expected reward from state s

10.2 Action Value Function

Q(s, a): Expected reward from taking action a in state s

Q(s, a) is more detailed but harder to compute.

11. Types of RL Problems

Episodic Tasks

- Have a clear end (e.g., games, maze)

Continuing Tasks

- No end (e.g., robot walking)

12. Applications of RL

- Robotics
- Game playing (AlphaGo)
- Autonomous driving
- Recommendation systems

13. Advantages

- Does not require labeled data
- Learns optimal behavior
- Handles sequential decision making

Getting Lost Example (Important Illustration)

This example explains RL using a real-world scenario.

Problem Description

You are lost in a city with squares labeled A–F.
Goal: Reach square **F** (backpacker's hostel).

State Representation

Each square = one state:

A, B, C, D, E, F

Environment Diagram

A —— B —— C

| |

D —— E —— F

- Nodes = States
- Edges = Possible actions (movements)
- F = Goal (absorbing state)

Actions

- Move between connected squares
- Stay in same square

Reward Design

- Reaching F → +100
- Staying in same place → -5
- Other moves → 0

Key Learning Idea

Agent explores randomly at first, then learns optimal path using rewards.

Optimal Policy Path

A → B → C → F

Markov Property and Markov Decision Process (MDP)

Markov Property

Future depends only on present state:

$P(s_{t+1} \mid s_t, a_t)$

Diagram: Markov Property

Past States → s_{t-2} → s_{t-1} → [s_t] → s_{t+1}

↑

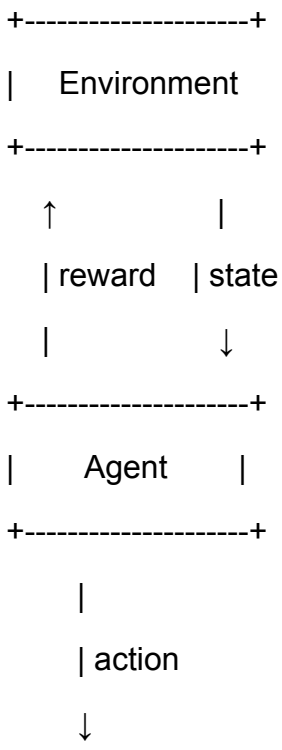
Decision depends only on s_t

Markov Decision Process (MDP)

Defined by:

- States (S)
- Actions (A)
- Transition probabilities (P)
- Reward function (R)
- Discount factor (γ)

MDP Flow Diagram



Markov Chain Monte Carlo (MCMC) Methods

1 Markov Chain

A sequence where next state depends only on current state.

Diagram: Markov Chain

S1 → S2 → S3 → S4 → S5 → ...

Each transition follows:

P(S_{t+1} | S_t)

2 Monte Carlo Method

Uses random sampling to approximate results.

Example:

- Estimating probability distributions

3 MCMC Concept

Combines both:

- Build a Markov chain
- Use random sampling
- Converges to target distribution

Diagram: MCMC Sampling Process

Start → x1 → x2 → x3 → x4 → ... → xn

(Markov transitions)

After many steps:

Samples ≈ Target Distribution P(x)

Sampling in MCMC

Process Steps

1. Initialize x0

2. Propose x'

3. Accept/Reject

4. Move to next state

5. Repeat

Proposal Distribution

Defines how new samples are generated:

Q(x' | x)

Diagram

Current State x

|

| sample from Q(x'|x)

↓

Proposed State x'

Metropolis-Hastings Algorithm

Steps

Current x

|

| propose x'

↓

Compute acceptance α

|

| accept with probability α

↓

Next state

Acceptance probability:

α = min(1, P(x')Q(x|x') / P(x)Q(x'|x))

Gibbs Sampling

Idea

Sample one variable at a time.

Diagram

(x1, x2, x3)

↓

sample x1 | x2,x3

↓

sample x2 | x1,x3

↓

sample x3 | x1,x2

↓

repeat...

Sampling

Sampling is the process of generating data points from a probability distribution.

In many real-world problems, the distribution P(x) is complex and direct sampling is not possible. Hence, approximate methods like MCMC are used.

Why Sampling?

- Bayesian inference
- Graphical models
- Hidden variable models
- Numerical integration

Types of Sampling

- Direct Sampling: From simple distributions (e.g., Gaussian)

- Rejection Sampling: Accept/reject samples
- MCMC Sampling: Uses Markov chains

MCMC Sampling Concept

Instead of sampling directly from P(x), we construct a Markov chain:

x0 → x1 → x2 → x3 → ... → xn

After many iterations, samples follow the target distribution.

Proposal Distribution

Proposal distribution defines how a new sample is proposed from the current state:

Q(x' | x)

Where:

- x = current state
- x' = proposed next state

Diagram

Current State (x)

|

| sample using Q(x'|x)

↓

Proposed State (x')

Intuition

Proposal distribution answers:
"Where should we move next?"

Importance of Proposal Distribution

- Direct sampling from P(x) is difficult

- Proposal distribution enables exploration
- Works with acceptance step to ensure correctness

Properties of Good Proposal Distribution

- Should explore entire state space
- Should not make very large jumps
- Should not move too slowly

Trade-off

- Large jumps → low acceptance rate
- Small steps → slow convergence

Common Proposal Distributions

1. Gaussian Proposal:

$x' = x + N(0, \sigma^2)$

2. Uniform Proposal:

$x' \sim U(a, b)$

3. Symmetric Proposal:

$Q(x'|x) = Q(x|x')$

Role in Metropolis-Hastings

Proposal distribution is used to compute acceptance probability:

$\alpha = \min(1, P(x')Q(x|x') / P(x)Q(x'|x))$

Special Case (Symmetric Proposal)

If $Q(x'|x) = Q(x|x')$:

$\alpha = \min(1, P(x') / P(x))$

Sampling Algorithm Flow

1. Initialize x
2. Generate x' using Q(x'|x)
3. Compute acceptance probability α
4. Accept or reject x'
5. Repeat

Markov Chain Monte Carlo (MCMC)

Markov Chain Monte Carlo (MCMC) is a class of algorithms used to sample from complex probability distributions where direct sampling is difficult.

It combines:

- Markov Chains (memoryless transitions)
- Monte Carlo Methods (random sampling)

Key Idea

Construct a Markov chain such that its stationary distribution is the target distribution P(x).

After many transitions, the generated samples approximate P(x).

Diagram: MCMC Process

Start → x1 → x2 → x3 → x4 → ... → xn

(Markov transitions)

After convergence:

Samples ≈ Target Distribution P(x)

Working Principle

1. Start from an initial state x0
2. Propose a new state using proposal distribution
3. Accept or reject the new state

4. Move to next state
5. Repeat many times

Important Algorithms

- Metropolis-Hastings
- Gibbs Sampling

Applications

- Bayesian inference
- Statistical physics
- Machine learning models

Graphical Models

1 Introduction

Graphical Models are probabilistic models that use graphs to represent dependencies between random variables.

They combine:

- Graph theory
- Probability theory

2 Components

- Nodes → Random variables
- Edges → Dependencies

3 Types of Graphical Models

1. Bayesian Networks (Directed)
2. Markov Random Fields (Undirected)

4 Bayesian Network (Directed Graph)

Represents conditional dependencies using directed edges.

Diagram

A → B → C

↓

D

Interpretation:

- A influences B
- B influences C and D

Joint distribution:

P(A, B, C, D) = P(A)P(B|A)P(C|B)P(D|B)

5 Markov Random Field (Undirected Graph)

Represents relationships without direction.

Diagram

A — B — C

| |

D — E — F

- Edges represent mutual dependence
- No direction of influence

6 Conditional Independence

Graphical models help identify independence relationships.

Example:
If A → B → C, then:
$A \perp C \mid B$ (A is independent of C given B)

7 Role of MCMC in Graphical Models

In complex graphical models:

- Exact inference is difficult

- MCMC is used for approximate inference

Steps:

1. Sample variables using MCMC
2. Estimate probabilities
3. Perform inference

Diagram: MCMC in Graphical Models

Graphical Model (Nodes & Edges)

↓

Apply MCMC Sampling

↓

Generate Samples

↓

Estimate Probabilities

Key Points

- MCMC is used for sampling complex distributions
- Graphical models represent variable dependencies
- Bayesian Networks → directed
- Markov Random Fields → undirected
- MCMC helps perform inference in graphical models

Markov Chain Monte Carlo (MCMC) – Graphical Models

Markov Chain Monte Carlo (MCMC)

1 Definition

Markov Chain Monte Carlo (MCMC) is a class of algorithms used to **generate samples from complex probability distributions** where direct sampling is difficult.

It combines:

- **Markov Chain** → next state depends only on current state
- **Monte Carlo** → random sampling

2 Key Idea

Construct a **Markov chain** such that:

[
\text{Stationary Distribution} = P(x)
]

After many iterations:

- Generated samples approximate the target distribution.

3 MCMC Process (Diagram)

$x \rightarrow x \rightarrow x \rightarrow x \rightarrow x \rightarrow \ldots \rightarrow x_n$

(Markov transitions)

After many steps:

Samples ≈ Target Distribution P(x)

4 Working Steps

1. Start with initial state x
2. Propose new state x'
3. Accept or reject x'
4. Move to next state
5. Repeat many times

5 Applications

- Bayesian inference
- Statistical modeling

- Graphical models
- Machine learning

Graphical Models

2.1 Definition

Graphical Models are probabilistic models that represent **random variables and their dependencies using graphs**.

2.2 Components

- **Nodes** → Random variables
- **Edges** → Dependencies

2.3 Types of Graphical Models

1. **Bayesian Networks (Directed)**
2. **Markov Random Fields (Undirected)**

3. Bayesian Networks (Directed Graphical Model)

3.1 Concept

- Uses **directed edges (→)**
- Represents **conditional dependencies**

3.2 Diagram

A → B → C

↓

D

3.3 Interpretation

- A influences B
- B influences C and D

3.4 Joint Probability

$$
P(A, B, C, D) = P(A), P(B|A), P(C|B), P(D|B)
$$

4. Markov Random Fields (Undirected Graphical Model)

4.1 Concept

- Uses **undirected edges (—)**
- Represents **mutual dependence**

4.2 Diagram

A — B — C

| |

D — E — F

4.3 Interpretation

- No direction of influence
- Variables influence each other mutually

5. Conditional Independence

Graphical models help identify independence relationships.

Example:

A → B → C

$$
A \perp C \mid B
$$

A is independent of C given B.

6. Role of MCMC in Graphical Models

6.1 Problem

- Exact inference is difficult in large graphical models
- Computing probabilities becomes complex

6.2 Solution: MCMC

MCMC is used for **approximate inference**.

6.3 Process Diagram

Graphical Model (Nodes & Edges)

↓

Apply MCMC Sampling

↓

Generate Samples

↓

Estimate Probabilities

Bayesian Networks (Directed Graphical Models)

1. Definition

A **Bayesian Network (BN)** is a probabilistic graphical model that represents a set of random variables and their **conditional dependencies using a Directed Acyclic Graph (DAG)**.

- **Nodes** → Random variables
- **Directed edges (→)** → Conditional dependencies

2. Key Properties

- The graph is **directed**
- The graph is **acyclic** (no loops)

- Each node depends only on its **parent nodes**
- Encodes **joint probability distribution** efficiently

3. Structure of Bayesian Network

Diagram

A → B → C

↓

D

Interpretation

- A influences B
- B influences C and D
- Dependencies are directional

4. Joint Probability Representation

A Bayesian Network factorizes the joint distribution as:

[
P(A, B, C, D) = P(A), P(B|A), P(C|B), P(D|B)
]

✓ Instead of computing full joint distribution, we use **local conditional probabilities**

5. Conditional Independence

Bayesian Networks encode independence relationships.

Example

A → B → C

[
A \perp C \mid B
]

☞ A is independent of C given B

6. Components of Bayesian Network

1. **Graph Structure (DAG)**
    - Defines relationships
2. **Conditional Probability Tables (CPTs)**
    - Defines probabilities

Example CPT

For node B:

A	P(B|A)
0	0.3
1	0.8

7. Inference in Bayesian Networks

Inference means computing probabilities such as:

$$
P(X \mid evidence)
$$

Types of Inference

- Exact inference (Variable elimination)
- Approximate inference (Sampling, MCMC)

8. Learning in Bayesian Networks

8.1 Structure Learning

- Learn graph from data

8.2 Parameter Learning

- Learn CPT values

9. Applications

- Medical diagnosis
- Fault detection systems

- Speech recognition
- Machine learning models
- Decision support systems

10. Advantages

- Compact representation of joint probability
- Handles uncertainty effectively
- Clear visualization of dependencies

11. Limitations

- Structure learning is complex
- Inference can be computationally expensive
- Requires domain knowledge

Markov Random Fields (MRF) – Undirected Graphical Models

1. Definition

A **Markov Random Field (MRF)** is a probabilistic graphical model that represents a set of random variables using an **undirected graph**.

- **Nodes** → Random variables
- **Edges (—)** → Mutual dependencies

Unlike Bayesian Networks, MRFs **do not have direction** in relationships.

2. Key Properties

- Graph is **undirected**
- Represents **symmetric relationships**
- Based on **Markov Property**
- A node depends only on its **neighbors**

3. Structure of MRF

Diagram

A — B — C

| |

D — E — F

Interpretation

- Each variable interacts with its neighboring nodes
- No direction of influence
- Dependencies are mutual

4. Markov Property (Local Independence)

A node is conditionally independent of all other nodes given its neighbors.

Example

For node **B**:

A — B — C

$$
B \perp \text{others} \mid (A, C)
$$

☞ B depends only on A and C

5. Factorization of Joint Distribution

In MRFs, the joint probability is expressed using **cliques** (fully connected subgraphs):

$$
P(X) = \frac{1}{Z} \prod_{c \in \text{cliques}} \psi_c(X_c)
$$

Where:

- ψ_c = potential function
- Z = normalization constant (partition function)

6. Clique Concept

A **clique** is a subset of nodes where every pair is connected.

Example

A — B

| \ |

C — D

- {A, B, C, D} forms a clique

7. Potential Functions

- Measure compatibility between variables
- Not probabilities directly
- Used to compute joint distribution

8. Inference in MRF

Goal: Compute probabilities such as:

[
P(X | evidence)
]

Methods

- Exact inference (Belief propagation)
- Approximate inference (MCMC, Gibbs sampling)

9. Applications

- Image processing (denoising, segmentation)
- Computer vision
- Natural language processing
- Spatial data modeling

10. Advantages

- Models symmetric relationships well

- Suitable for spatial problems
- Flexible representation

11. Limitations

- Partition function (Z) is hard to compute
- Inference can be expensive
- Parameter learning is complex

Hidden Markov Models (HMM)

1. Definition

A **Hidden Markov Model (HMM)** is a statistical model where the system is assumed to be a **Markov process with hidden (unobserved) states**.

- We **cannot directly observe states**
- We only observe **outputs (observations)** generated from those states

2. Key Idea

- The system moves between hidden states
- Each state produces an observable output
- We try to **infer hidden states from observations**

3. Structure of HMM

Diagram

Hidden States: S1 → S2 → S3 → S4

↓ ↓ ↓ ↓

Observations: O1 O2 O3 O4

Interpretation

- (S_t) = hidden state at time t
- (O_t) = observation at time t
- Transitions occur between hidden states
- Observations depend on current state

4. Components of HMM

An HMM is defined by:

4.1 States (S)

- Hidden states of the system
- Example: Weather (Sunny, Rainy)

4.2 Observations (O)

- Visible outputs
- Example: Umbrella usage

4.3 Transition Probabilities (A)

[
A = P(S_{t+1} | S_t)
]

Probability of moving from one state to another

4.4 Emission Probabilities (B)

[
B = P(O_t | S_t)
]

Probability of observation given a state

4.5 Initial Probabilities (π)

[
\pi = P(S_1)

]

Probability of starting state

5. HMM Representation

Complete model is defined as:

[
\lambda = (A, B, \pi)
]

6. Example (Weather Model)

States:

- Sunny, Rainy

Observations:

- Umbrella, No Umbrella

Diagram

Sunny ↔ Rainy

↓ ↓

Umbrella No Umbrella

7. Key Problems in HMM (Very Important ⭐)

7.1 Evaluation Problem

Find probability of observation sequence:

[
P(O | \lambda)
]

Solution: **Forward Algorithm**

7.2 Decoding Problem

Find most likely hidden state sequence:

[
S_1, S_2, ..., S_T

]

Solution: **Viterbi Algorithm**

7.3 Learning Problem

Estimate model parameters (A, B, π)

Solution: **Baum-Welch Algorithm (EM)**

8. Independence Assumptions

1. **Markov Property**:
   [
   P(S_t | S_{t-1}, ..., S_1) = P(S_t | S_{t-1})
   ]
2. **Output Independence**:
   [
   P(O_t | S_t, \text{past}) = P(O_t | S_t)
   ]

9. Applications

- Speech recognition
- Natural language processing
- Bioinformatics (DNA sequencing)
- Part-of-speech tagging
- Activity recognition

10. Advantages

- Handles sequential data
- Models hidden processes
- Efficient algorithms available

11. Limitations

- Assumes Markov property (simplification)

- Limited to simple dependencies
- Not suitable for complex long-range patterns

Tracking Methods (in Machine Learning & Probabilistic Models)

1. Definition

Tracking methods are techniques used to **estimate and follow the state of a system over time** using sequential observations.

- The true state is often **hidden or noisy**
- Observations are used to **infer and update the state**

2. Key Idea

Tracking involves:

☞ Estimating the current state
☞ Updating the estimate as new data arrives

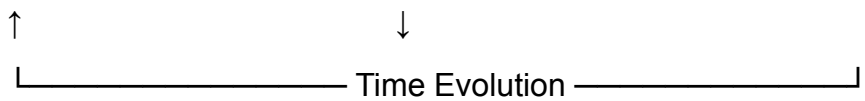
Used in:

- Object tracking (vision)
- Robotics (position tracking)
- Navigation systems

3. Basic Tracking Framework

Diagram

True State (hidden) → Observation → Estimation → Updated State



4. Components of Tracking

1. **State (x)**
    - True condition of system (e.g., position, velocity)

2. **Observation (z)**
    - Measured data (noisy)
3. **Transition Model**

[
P(x_t | x_{t-1})
]

- How state evolves over time

4. **Observation Model**

[
P(z_t | x_t)
]

- How observations relate to state

5. Types of Tracking Methods

5.1 Kalman Filter (Linear Tracking)

Concept

- Used when system is **linear + Gaussian noise**

Diagram

Prediction → Update → Prediction → Update → ...

Steps

1. Predict next state
2. Update using observation

Features

- Efficient
- Optimal for linear systems

5.2 Extended Kalman Filter (EKF)

Concept

- Handles **nonlinear systems**
- Uses linear approximation

5.3 Particle Filter (Sequential Monte Carlo)

Concept

- Uses **set of samples (particles)** to represent state distribution

Diagram

Particles: • • • • • • • •

↓ ↓ ↓ ↓ ↓ ↓ ↓

Move + Weight + Resample

Steps

1. Initialize particles
2. Predict movement
3. Assign weights
4. Resample

Features

- Works for nonlinear, non-Gaussian systems
- More flexible than Kalman filter

5.4 Hidden Markov Model (HMM) Tracking

Concept

- Tracks hidden states over time
- Uses probabilistic transitions

Diagram

S1 → S2 → S3 → S4

↓ ↓ ↓ ↓

O1 O2 O3 O4

6. General Tracking Process

Initialize State

↓

Predict Next State

↓

Observe Data

↓

Update Estimate

↓

Repeat

7. Applications

- Object tracking in videos
- GPS navigation
- Robotics localization
- Radar and surveillance systems
- Human activity tracking

8. Challenges

- Noisy observations
- Hidden states
- Nonlinear systems
- Computational complexity